



Manejo de múltiples respuestas posibles

Una app real **no responde siempre lo mismo**. Un sistema suele tener:

- casos válidos
- casos inválidos (con errores concretos)
- casos "parciales" o "desconocidos"
- varios tipos de tarea que requieren respuestas diferentes

La habilidad de esta sección es:

controlar el flujo y decidir **qué respuesta dar** según el caso.

Objetivos

- Devolver respuestas estructuradas.
- Controlar el flujo según validación y tipo de tarea.
- Comparar `if/elif/else` vs tabla de handlers (`dict`).
- Separar la lógica de la presentación

Esto se usará para nuestros proyectos de IA en el futuro:

- Integraciones con LLM
 - Contexto e historial
 - Para dotar de estructura a proyectos
-

1) Patrón recomendado: salida estructurada

En vez de devolver solo strings, devuelve un dict. Para empezar, con **dos estados** es suficiente:

```
{  
  "status": "ok" | "error",  
  "mensaje": "...",  
}
```

```
"data": {...} # opcional (puedes ignorarlo al principio)
}
```

Ejemplo de helpers para estandarizar las respuestas:

```
def respuesta_ok(mensaje: str, data: dict | None = None) -> dict:
    return {"status": "ok", "mensaje": mensaje, "data": data or {}}

def respuesta_error(mensaje: str, errores: list[str]) -> dict:
    return {"status": "error", "mensaje": mensaje, "data": {"errores": errores}}
```

Ventajas:

- el `main` puede decidir qué imprimir / mostrar
- puedes testear la lógica sin depender de prints
- no acoplas "procesar" con "presentar"

2) Responder distinto según el resultado de validación

Un flujo típico:

- si hay errores → respuesta `error` con lista de errores
- si está OK → respuesta `ok` y continuar con la tarea

Ejemplo conceptual:

```
errores = validar_registro(datos)
if errores:
    return {"status": "error", "mensaje": "Input inválido", "data": {"errores":
errores}}
return {"status": "ok", "mensaje": "Registro válido", "data": datos}
```

Ejemplo más claro usando los helpers:

```
def procesar_registro(datos: dict) -> dict:
    errores = validar_registro(datos)
    if errores:
        return respuesta_error("Input inválido", errores)

    # Aquí todavía no hay LLM: solo devolvemos "qué haríamos"
    return respuesta_ok(
        "Registro válido",
        data={"tipo": datos.get("tipo"), "prioridad": datos.get("prioridad")},
    )
```

3) Responder distinto según el “tipo de tarea”

Si el input tiene un campo `tipo` (por ejemplo `summary`, `translate`, `qa`), tu app puede devolver:

- un mensaje distinto
- un formato distinto
- datos distintos

3.1 `if / elif / else`

```
def responder_por_tipo_if(tipo: str) -> dict:
    t = (tipo or "").strip().lower()

    if t == "summary":
        return respuesta_ok("Haré un resumen.", data={"tipo": t})
    elif t == "translate":
        return respuesta_ok("Haré una traducción.", data={"tipo": t})
    elif t == "qa":
        return respuesta_ok("Haré preguntas y respuestas.", data={"tipo": t})
    else:
        return respuesta_error("Tipo desconocido", [f"tipo={t!r}"])
```

3.2 “Tabla de handlers” con `dict`

Buena cuando hay muchas opciones.

Si esto te parece demasiado al principio: quédate con `if/elif/else` y vuelve aquí cuando ya lo tengas claro.

```
HANDLERS = {
    "summary": handler_summary,
    "translate": handler_translate,
    "qa": handler_qa,
}
```

Y luego:

```
handler = HANDLERS.get(tipo)
if handler is None:
    return {"status": "error", "mensaje": "Tipo desconocido", "data": {"tipo":
tipo}}
return handler(datos)
```

Ejemplo completo con handlers (refactor del `if/elif/else`):

```
def handler_summary(datos: dict) -> dict:
    return respuesta_ok("Haré un resumen.", data={"input": datos})

def handler_translate(datos: dict) -> dict:
    return respuesta_ok("Haré una traducción.", data={"input": datos})

def handler_qa(datos: dict) -> dict:
    return respuesta_ok("Haré preguntas y respuestas.", data={"input": datos})

HANDLERS = {
    "summary": handler_summary,
    "translate": handler_translate,
    "qa": handler_qa,
}

def responder_con_handlers(datos: dict) -> dict:
    tipo = (datos.get("tipo") or "").strip().lower()
    handler = HANDLERS.get(tipo)
    if handler is None:
        return respuesta_error("Tipo desconocido", [f"tipo={tipo!r}"])
    return handler(datos)
```

4) Resumen

- No devuelvas "texto suelto" si luego necesitas tomar decisiones.
- Separa: lógica (decisión) vs presentación (print).
- Prepara "tipos de respuesta" antes de meter un LLM.

Ejemplo de presentación separada (para el `main` o el notebook):

```
def imprimir_respuesta(r: dict) -> None:
    status = r.get("status", "unknown").upper()
    mensaje = r.get("mensaje", "")
    print(f"[{status}] {mensaje}")

    if r.get("status") == "error":
        for e in r.get("data", {}).get("errores", []):
            print(" -", e)
```